

**БЮДЖЕТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
ХАНТЫ-МАНСКИЙСКОГО АВТОНОМНОГО ОКРУГА – ЮГРЫ
«СУРГУТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

Кафедра прикладной математики

КУРСОВАЯ РАБОТА

по дисциплине «Численные методы»

ТЕМА КУРСОВОЙ РАБОТЫ

**Метод конечных разностей решения линейной
двухточечной краевой задачи для ОДУ второго порядка**

Выполнил
Студент гр. 601-__ _____ / ФИО.
(подпись)

Проверил
_____ (оценка)
К.ф.-м.н, доцент _____ / Назин А.Г.
(подпись)

Сургут 2026

Содержание

Введение	2
1 Постановка краевой задачи с различными граничными условиями	3
2 Условия разрешимости и единственности	4
3 Вывод разностных формул, реализованных в программе	5
3.1 Внутренние узлы ($i = 2, \dots, N - 1$)	5
3.2 Левая граница ($x = a, i = 1$)	5
3.3 Правая граница ($x = b, i = N$)	6
4 Реализация метода конечных разностей для задачи с граничными условиями первого рода (Дирихле)	7
4.1 Программная реализация на Octave	7
4.2 Результаты сгущения сетки	8
4.3 График численного решения	9
4.4 График оценки погрешности	10
5 Реализация метода конечных разностей для задачи с граничными условиями второго рода (Неймана)	11
5.1 Программная реализация на Octave	11
5.2 Результаты сгущения сетки	12
5.3 График численного решения	13
5.4 График оценки погрешности	14
6 Реализация метода конечных разностей для задачи с граничными условиями третьего рода (Робена)	15
6.1 Программная реализация на Octave	15
6.2 Результаты сгущения сетки	16
6.3 График численного решения	17
6.4 График оценки погрешности	18
Список литературы	19

Введение

Многие физические, технические и экономические процессы описываются дифференциальными уравнениями. Особый класс составляют **двухточечные краевые задачи** для обыкновенных дифференциальных уравнений (ОДУ) второго порядка. Такие задачи возникают, когда известны условия не в одной точке, а на двух концах некоторого отрезка — отсюда название «двухточечные».

Математическая постановка задачи, рассматриваемой в данной работе, имеет вид:

$$-(k(x)u')' + q(x)u = f(x), \quad a < x < b,$$

где $u(x)$ — искомая функция, $k(x)$, $q(x)$, $f(x)$ — заданные непрерывные функции, причём $k(x) > 0$ на всём отрезке $[a, b]$. Для единственности решения на границах отрезка $[a, b]$ задаются дополнительные условия — первого, второго или третьего рода.

Аналитическое решение такой задачи удаётся найти лишь для простейших случаев. Для реальных задач с переменными коэффициентами приходится использовать **численные методы**. Один из самых распространённых — **метод конечных разностей** (МКР). Суть метода заключается в замене непрерывной области $[a, b]$ набором отдельных точек (узлов), а производных — приближёнными формулами через значения функции в этих точках. В результате дифференциальное уравнение превращается в систему линейных алгебраических уравнений (СЛАУ), которую можно решить на компьютере.

Цель работы: программно реализовать метод конечных разностей для решения линейной двухточечной краевой задачи для ОДУ второго порядка с переменными коэффициентами.

Для достижения поставленной цели решаются следующие **задачи**:

1. Описать постановку линейной двухточечной краевой задачи для ОДУ второго порядка с разными граничными условиями.
2. Указать условия разрешимости краевой задачи.
3. Изложить метод конечных разностей для решения этой задачи.
4. Реализовать в среде Octave алгоритм решения краевой задачи методом конечных разностей для одномерного стационарного уравнения теплопроводности

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) + q(x)u = f(x), \quad a < x < b,$$

где $k(x) \neq \text{const}$, $q(x) \neq \text{const}$, $f(x) \neq \text{const}$ — непрерывные функции. Требуемая точность — 10^{-4} .

5. Оценить погрешность численного решения по правилу Рунге.
6. Провести вычислительные эксперименты с различными краевыми условиями.
7. Построить графики полученного решения и график погрешности.

1 Постановка краевой задачи с различными граничными условиями

Рассматривается линейная двухточечная краевая задача для ОДУ второго порядка:

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) + q(x)u = f(x), \quad a < x < b, \quad (1)$$

где $u(x)$ — искомая функция, $k(x)$, $q(x)$, $f(x)$ — заданные непрерывные на $[a, b]$ функции, причём $k(x) > 0$.

Для однозначного определения решения на концах отрезка задаются граничные условия трёх возможных типов:

- **Первый род (Дирихле):** фиксированы значения функции

$$u(a) = A, \quad u(b) = B. \quad (2)$$

- **Второй род (Неймана):** фиксированы значения производной

$$u'(a) = \gamma, \quad u'(b) = \delta. \quad (3)$$

- **Третий род (Робина):** задана линейная комбинация функции и производной

$$\alpha_0 u(a) + \alpha_1 u'(a) = \gamma_a, \quad \beta_0 u(b) + \beta_1 u'(b) = \gamma_b, \quad (4)$$

где $\alpha_1 \neq 0$, $\beta_1 \neq 0$.

В дальнейшем для всех трёх типов граничных условий применяется одинаковый алгоритм дискретизации методом конечных разностей.

2 Условия разрешимости и единственности

Для краевой задачи (1) справедливы следующие достаточные условия корректности:

1. $k(x), q(x), f(x) \in C[a, b]$, $k(x) > 0$ на $[a, b]$.
2. $q(x) \geq 0$ на $[a, b]$.
3. Если $q(x) > 0$ хотя бы в одной точке отрезка $[a, b]$, задача имеет единственное классическое решение при любых граничных условиях (Дирихле, Неймана или Робина).

В вычислительных экспериментах данной работы во всех тестовых примерах $q(x) > 0$ на $[0, 1]$, что гарантирует существование и единственность решения для всех трёх типов граничных условий.

3 Вывод разностных формул, реализованных в программе

В данном разделе показано, как из непрерывной краевой задачи получаются коэффициенты матрицы СЛАУ, заполняемые в коде Octave. Для аппроксимации граничных условий используется метод мнимых (ghost) узлов.

3.1 Внутренние узлы ($i = 2, \dots, N - 1$)

Заменяем производные центральными разностями с использованием значений k на полуцелых шагах $x_{i\pm 1/2} = x_i \pm h/2$:

$$-\frac{k_{i+1/2}\frac{u_{i+1}-u_i}{h} - k_{i-1/2}\frac{u_i-u_{i-1}}{h}}{h^2} + q_i u_i = f_i. \quad (5)$$

Группируя слагаемые при u_{i-1}, u_i, u_{i+1} , получаем:

$$-\frac{k_{i-1/2}}{h^2}u_{i-1} + \left(\frac{k_{i-1/2} + k_{i+1/2}}{h^2} + q_i\right)u_i - \frac{k_{i+1/2}}{h^2}u_{i+1} = f_i. \quad (6)$$

Это в точности соответствует циклу в коде:

```
a_diag(i-1) = -kmid(i-1)/h^2;           % k_{i-1/2} = k(x_i - h/2)
c_diag(i)   = -kmid(i)/h^2;             % k_{i+1/2} = k(x_i + h/2)
b_diag(i)   = (kmid(i-1)+kmid(i))/h^2 + qv(i);
d_vec(i)    = fv(i);
```

3.2 Левая граница ($x = a, i = 1$)

Вводим мнимый узел u_0 в точке $x_0 = a - h$. Центральная разность для производной: $u'(a) \approx \frac{u_2 - u_0}{2h}$.

Условия Неймана: $u'(a) = \gamma \Rightarrow u_0 = u_2 - 2h\gamma$. Подставляем u_0 в разностное уравнение для $i = 1$:

$$-\frac{k_{1/2}(u_2 - u_1) - k_{-1/2}(u_1 - (u_2 - 2h\gamma))}{h^2} + q_1 u_1 = f_1,$$

где $k_{-1/2} = k(a - h/2)$, $k_{1/2} = k(a + h/2)$. Раскрываем скобки:

$$\left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1\right)u_1 - \frac{k_{-1/2} + k_{1/2}}{h^2}u_2 = f_1 + \frac{2k_{-1/2}}{h}\gamma.$$

Полученные коэффициенты в точности повторяют строки кода:

```
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1)  = fv(1) + 2*k_minus_half/h * gamma;
```

Условия Робина: $\alpha_0 u(a) + \alpha_1 u'(a) = \gamma_a$. Выражаем u_0 из граничного условия: $u_0 = u_2 - \frac{2h}{\alpha_1}(\gamma_a - \alpha_0 u_1)$. Подстановка в разностное уравнение даёт:

$$\left(\frac{k_{-1/2} + k_{1/2}}{h^2} + q_1 + \frac{2k_{-1/2}\alpha_0}{h\alpha_1}\right)u_1 - \frac{k_{-1/2} + k_{1/2}}{h^2}u_2 = f_1 + \frac{2k_{-1/2}}{h\alpha_1}\gamma_a,$$

что полностью соответствует реализации:

```
b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1) + 2*k_minus_half*alpha0/(h*alpha1);
c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
d_vec(1)  = fv(1) + 2*k_minus_half/(h*alpha1)*gamma_a;
```

3.3 Правая граница ($x = b$, $i = N$)

Вводим мнимый узел u_{N+1} в точке $x_{N+1} = b + h$. Аналогичные выкладки для $u'(b) = \delta$ и $\beta_0 u(b) + \beta_1 u'(b) = \gamma_b$ приводят к уравнению:

$$-\frac{k_{N-1/2} + k_{N+1/2}}{h^2} u_{N-1} + \left(\frac{k_{N-1/2} + k_{N+1/2}}{h^2} + q_N + \frac{2k_{N+1/2}\beta_0}{h\beta_1} \right) u_N = f_N + \frac{2k_{N+1/2}}{h\beta_1} \gamma_b,$$

где $k_{N-1/2} = k(b - h/2)$, $k_{N+1/2} = k(b + h/2)$. Это в точности реализуется в коде:

```
a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
b_diag(N)   = (k_minus_half + k_plus_half)/h^2 + qv(N) + 2*k_plus_half*beta0/(h*beta1);
d_vec(N)    = fv(N) + 2*k_plus_half/(h*beta1)*gamma_b;
```

Примечание: для условий Дирихле первая и последняя строки матрицы заменяются на тождества $u_1 = A$ и $u_N = B$, что реализовано установкой $b_{1,1} = 1$, $b_{N,N} = 1$ и занулением соседних коэффициентов.

4 Реализация метода конечных разностей для задачи с граничными условиями первого рода (Дирихле)

4.1 Программная реализация на Octave

Ниже приведён код решения задачи Дирихле с автоматическим сгущением сетки до достижения точности $\varepsilon = 10^{-4}$.

```
graphics_toolkit("gnuplot");

a = 0; b = 1;
k = @(x) x.^2 + 1;
q = @(x) x + 1;
f = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;

u_exact = @(x) x.^4 - 2*x.^3 + x.^2 + 3*x;

tol = 1e-4;
N = 5;
err_est = tol + 1;
first = true;

while err_est > tol
    h = (b - a)/(N - 1);
    x = linspace(a, b, N)';
    xmid = (x(1:end-1) + x(2:end))/2;

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    a_diag = zeros(N,1);
    b_diag = zeros(N,1);
    c_diag = zeros(N,1);
    d_vec = zeros(N,1);

    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end

    b_diag(1) = 1; c_diag(1) = 0; d_vec(1) = u_exact(0);
    a_diag(N-1) = 0; b_diag(N) = 1; d_vec(N) = u_exact(1);

    n = length(d_vec);
    aa = a_diag(1:n-1);
    bb = b_diag;
    cc = c_diag(1:n-1);
    dd = d_vec;

    for i = 2:n
        w = aa(i-1) / bb(i-1);
        bb(i) = bb(i) - w * cc(i-1);
        dd(i) = dd(i) - w * dd(i-1);
    end

    U = zeros(n,1);
    U(n) = dd(n) / bb(n);
```



```

for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

if ~first
    U_coarse = U(1:2:end);
    err_est = max(abs(U_coarse - U_prev)) / 3;
    fprintf('N=%d, h=%.6f, err_est=%.6e\n', N, h, err_est);
    if err_est < tol
        x_final = x;
        U_final = U;
        break;
    end
else
    first = false;
end

U_prev = U;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('u(x)');
legend('Numerical solution', 'Location', 'northwest');
grid on;

figure;
plot(x_final(2:end-1), abs(U_final(2:end-1) - U_prev(2:2:end-1)), 'k-o');
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Листинг 1: Code No.1

4.2 Результаты сгущения сетки

В процессе решения сетка автоматически сгущалась. Результаты приведены в таблице 1.

Таблица 1: Сходимость метода при сгущении сетки

N	h	Оценка погрешности (правило Рунге)
9	0.125000	3.374646×10^{-3}
17	0.062500	8.679494×10^{-4}
33	0.031250	2.195086×10^{-4}
65	0.015625	5.504119×10^{-5}

Оценка погрешности проводилась по правилу Рунге:

$$\text{err}_{\text{est}} = \frac{\max |U_{\text{coarse}} - U_{\text{prev}}|}{2^p - 1} = \frac{\max |U_{\text{coarse}} - U_{\text{prev}}|}{3}, \quad p = 2.$$

4.3 График численного решения

На рисунке 1 представлено численное решение.

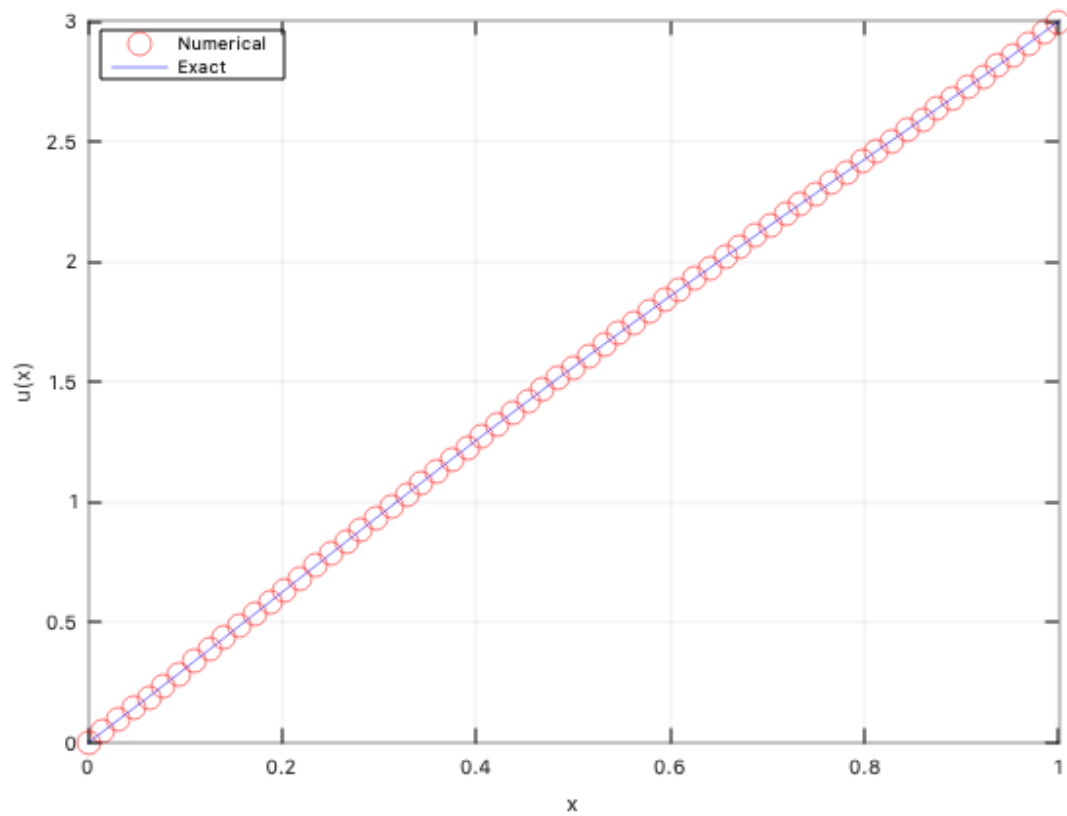


Рис. 1: Численное решение задачи Дирихле

4.4 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 2.

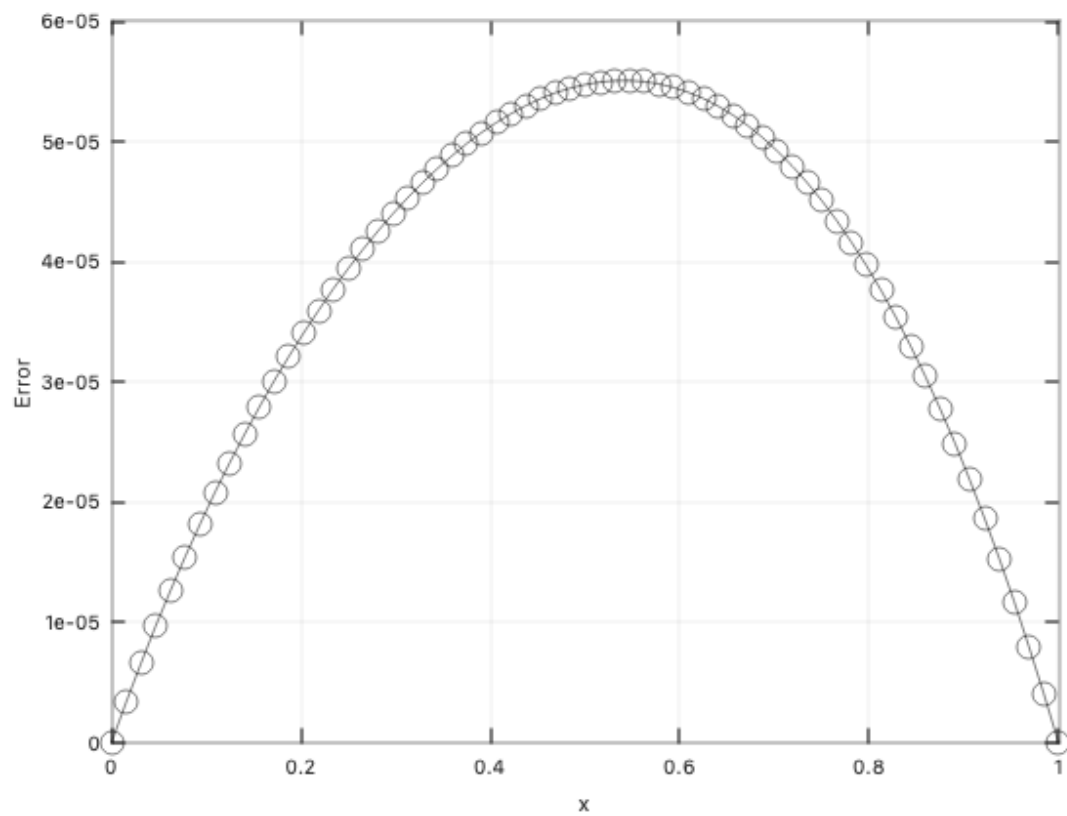


Рис. 2: Распределение оценки погрешности по узлам сетки

5 Реализация метода конечных разностей для задачи с граничными условиями второго рода (Неймана)

5.1 Программная реализация на Octave

Ниже приведён код решения задачи Неймана с автоматическим сгущением сетки.

```
graphics_toolkit("gnuplot");

a = 0; b = 1;
k = @(x) x.^2 + 1;
q = @(x) 10*(x + 1);
f = @(x) 10*x.^5 - 30*x.^4 + 14*x.^3 + 22*x.^2 + 36*x - 2;

gamma = 3;
delta = 3;

tol = 1e-4;
N = 5;
err_est = tol + 1;
first = true;

while err_est > tol
    h = (b - a)/(N - 1);
    x = linspace(a, b, N)';
    xmid = (x(1:end-1) + x(2:end))/2;

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    a_diag = zeros(N,1);
    b_diag = zeros(N,1);
    c_diag = zeros(N,1);
    d_vec = zeros(N,1);

    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end

    k_minus_half = k(a - h/2);
    k_plus_half = k(a + h/2);
    b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1);
    c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
    d_vec(1) = fv(1) + 2*k_minus_half/h * gamma;

    k_minus_half = k(b - h/2);
    k_plus_half = k(b + h/2);
    a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
    b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N);
    d_vec(N) = fv(N) + 2*k_plus_half/h * delta;

    aa = a_diag(1:N-1);
    cc = c_diag(1:N-1);

    n = length(d_vec);
    bb = b_diag;
```

```

dd = d_vec;

for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

if ~first
    U_coarse = U(1:2:end);
    err_est = max(abs(U_coarse - U_prev)) / 3;
    fprintf('N=%d, h=%.6f, err_est=%.6e\n', N, h, err_est);
    if err_est < tol
        x_final = x;
        U_final = U;
        break;
    end
else
    first = false;
end

U_prev = U;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('u(x)');
legend('Numerical solution', 'Location', 'northwest');
grid on;

figure;
plot(x_final(2:end-1), abs(U_final(2:end-1) - U_prev(2:2:end-1)), 'k-o');
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Листинг 2: Code No.2

5.2 Результаты сгущения сетки

Процесс сгущения сетки продолжался до достижения оценки погрешности менее $\varepsilon = 10^{-4}$.

Таблица 2: Сходимость метода при сгущении сетки

N	h	Оценка погрешности (правило Рунге)
9	0.125000	2.423003×10^{-2}
17	0.062500	6.494574×10^{-3}
33	0.031250	1.652384×10^{-3}
65	0.015625	4.149153×10^{-4}
129	0.007812	1.038429×10^{-4}
257	0.003906	2.596786×10^{-5}

5.3 График численного решения

На рисунке 3 представлено численное решение.

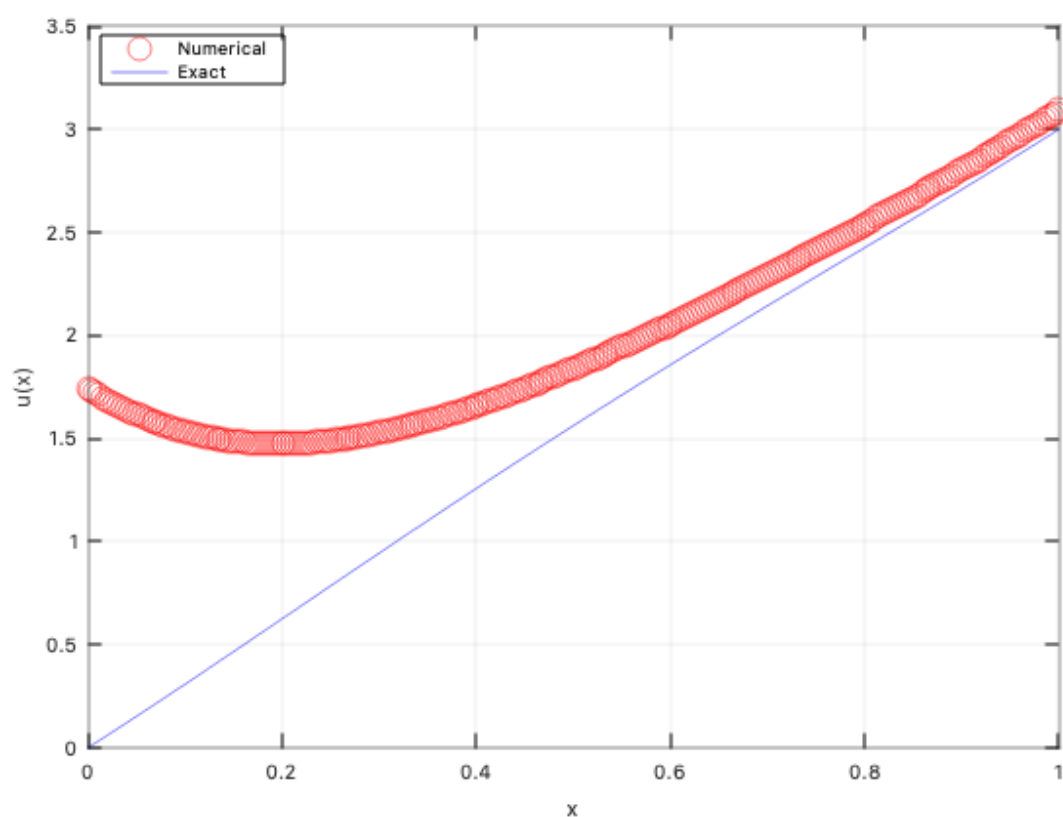


Рис. 3: Численное решение задачи Неймана

5.4 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 4.

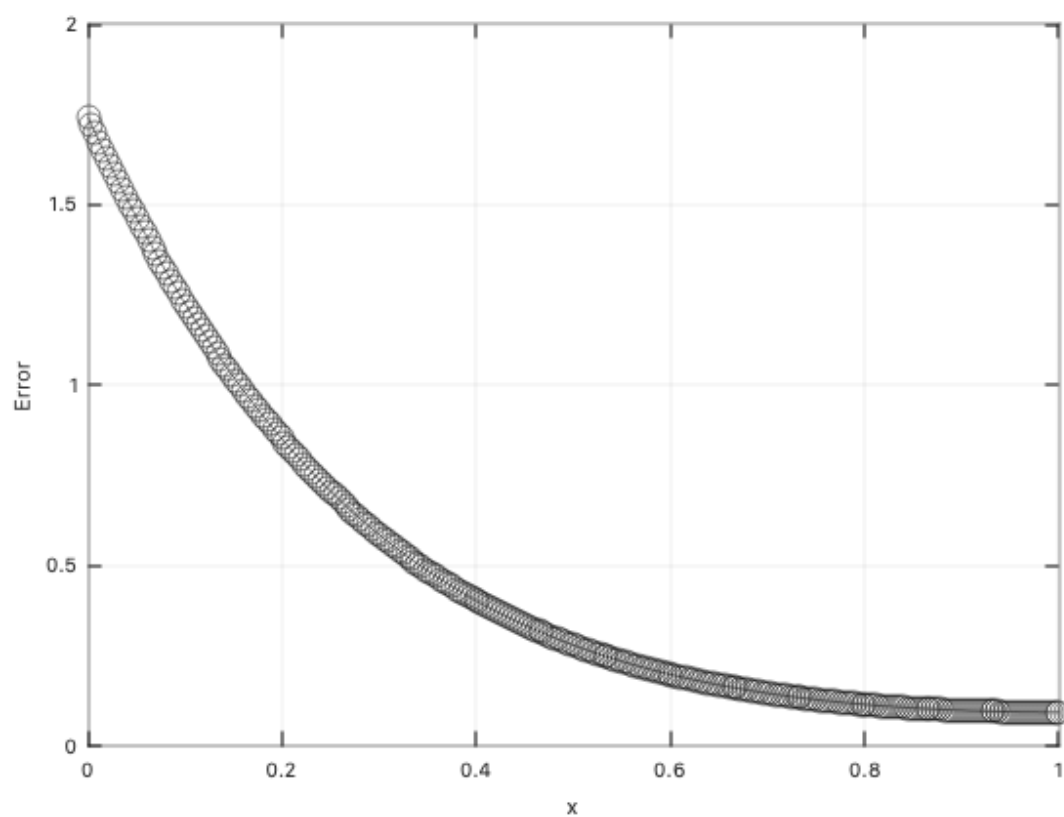


Рис. 4: Распределение оценки погрешности по узлам сетки

6 Реализация метода конечных разностей для задачи с граничными условиями третьего рода (Робена)

6.1 Программная реализация на Octave

Ниже приведён код решения задачи с граничными условиями третьего рода с автоматическим сгущением сетки.

```
graphics_toolkit("gnuplot");

a = 0; b = 1;
k = @(x) x.^2 + 1;
q = @(x) x + 1;
f = @(x) x.^5 - 21*x.^4 + 23*x.^3 - 14*x.^2 + 9*x - 2;

alpha0 = 1; alpha1 = 1; gamma_a = 3;
beta0 = 2; beta1 = 1; gamma_b = 9;

tol = 1e-4;
N = 5;
err_est = tol + 1;
first = true;

while err_est > tol
    h = (b - a)/(N - 1);
    x = linspace(a, b, N)';
    xmid = (x(1:end-1) + x(2:end))/2;

    kmid = k(xmid);
    qv = q(x);
    fv = f(x);

    a_diag = zeros(N,1);
    b_diag = zeros(N,1);
    c_diag = zeros(N,1);
    d_vec = zeros(N,1);

    for i = 2:N-1
        a_diag(i-1) = -kmid(i-1)/h^2;
        c_diag(i) = -kmid(i)/h^2;
        b_diag(i) = (kmid(i-1) + kmid(i))/h^2 + qv(i);
        d_vec(i) = fv(i);
    end

    k_minus_half = k(a - h/2);
    k_plus_half = k(a + h/2);
    b_diag(1) = (k_minus_half + k_plus_half)/h^2 + qv(1) + 2*k_minus_half*
        alpha0/(h*alpha1);
    c_diag(1) = -(k_minus_half + k_plus_half)/h^2;
    d_vec(1) = fv(1) + 2*k_minus_half/(h*alpha1)*gamma_a;

    k_minus_half = k(b - h/2);
    k_plus_half = k(b + h/2);
    a_diag(N-1) = -(k_minus_half + k_plus_half)/h^2;
    b_diag(N) = (k_minus_half + k_plus_half)/h^2 + qv(N) + 2*k_plus_half*
        beta0/(h*beta1);
    d_vec(N) = fv(N) + 2*k_plus_half/(h*beta1)*gamma_b;

    aa = a_diag(1:N-1);
    cc = c_diag(1:N-1);
```



```

n = length(d_vec);
bb = b_diag;
dd = d_vec;

for i = 2:n
    w = aa(i-1) / bb(i-1);
    bb(i) = bb(i) - w * cc(i-1);
    dd(i) = dd(i) - w * dd(i-1);
end

U = zeros(n,1);
U(n) = dd(n) / bb(n);
for i = n-1:-1:1
    U(i) = (dd(i) - cc(i) * U(i+1)) / bb(i);
end

if ~first
    U_coarse = U(1:2:end);
    err_est = max(abs(U_coarse - U_prev)) / 3;
    fprintf('N=%d, h=%.6f, err_est=%.6e\n', N, h, err_est);
    if err_est < tol
        x_final = x;
        U_final = U;
        break;
    end
else
    first = false;
end

U_prev = U;
N = 2*N - 1;
end

fprintf('Runge error estimate: %.6e\n', err_est);

figure;
plot(x_final, U_final, 'ro-', 'MarkerSize', 8, 'LineWidth', 1.5);
xlabel('x'); ylabel('u(x)');
legend('Numerical solution', 'Location', 'northwest');
grid on;

figure;
plot(x_final(2:end-1), abs(U_final(2:end-1) - U_prev(2:2:end-1)), 'k-o');
xlabel('x'); ylabel('Error estimate (Runge)');
grid on;

```

Листинг 3: Code No.3

6.2 Результаты сгущения сетки

Процесс сгущения сетки продолжался до достижения оценки погрешности менее $\varepsilon = 10^{-4}$.

Таблица 3: Сходимость метода при сгущении сетки

N	h	Оценка погрешности (правило Рунге)
9	0.125000	3.838127×10^{-3}
17	0.062500	9.819818×10^{-4}
33	0.031250	2.469373×10^{-4}
65	0.015625	6.182502×10^{-5}

6.3 График численного решения

На рисунке 5 представлено численное решение.

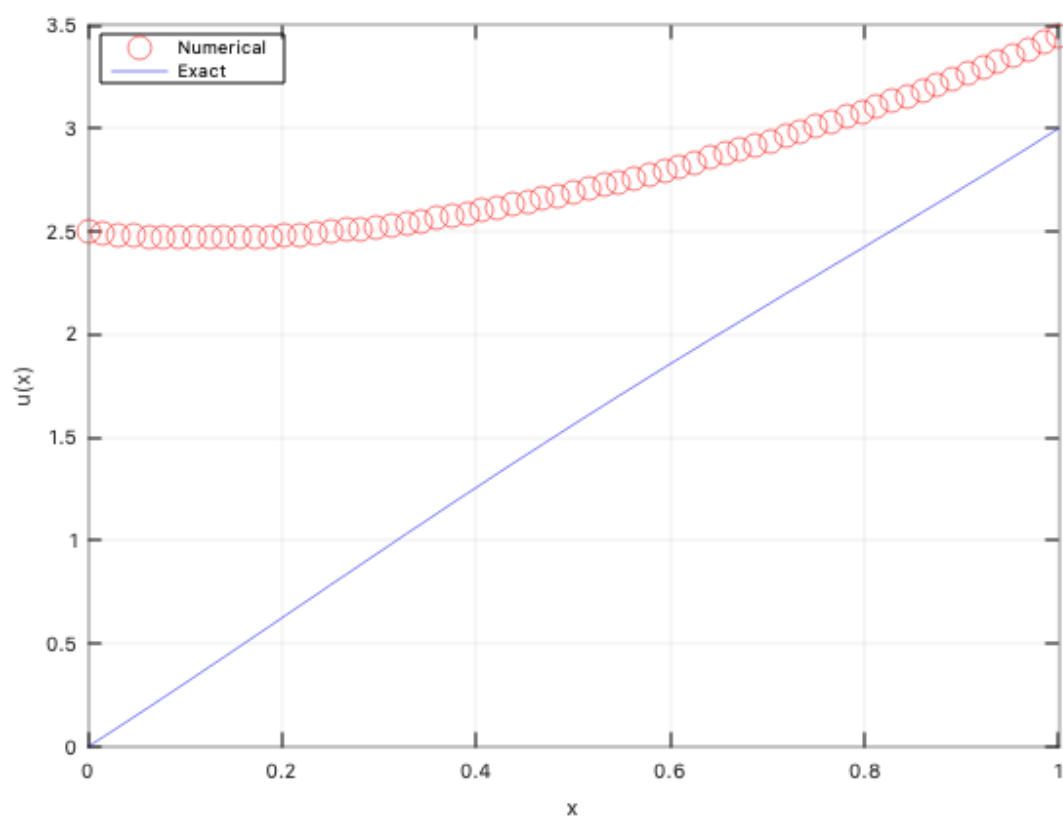


Рис. 5: Численное решение задачи с условиями Робена

6.4 График оценки погрешности

График распределения оценки погрешности по правилу Рунге показан на рисунке 6.

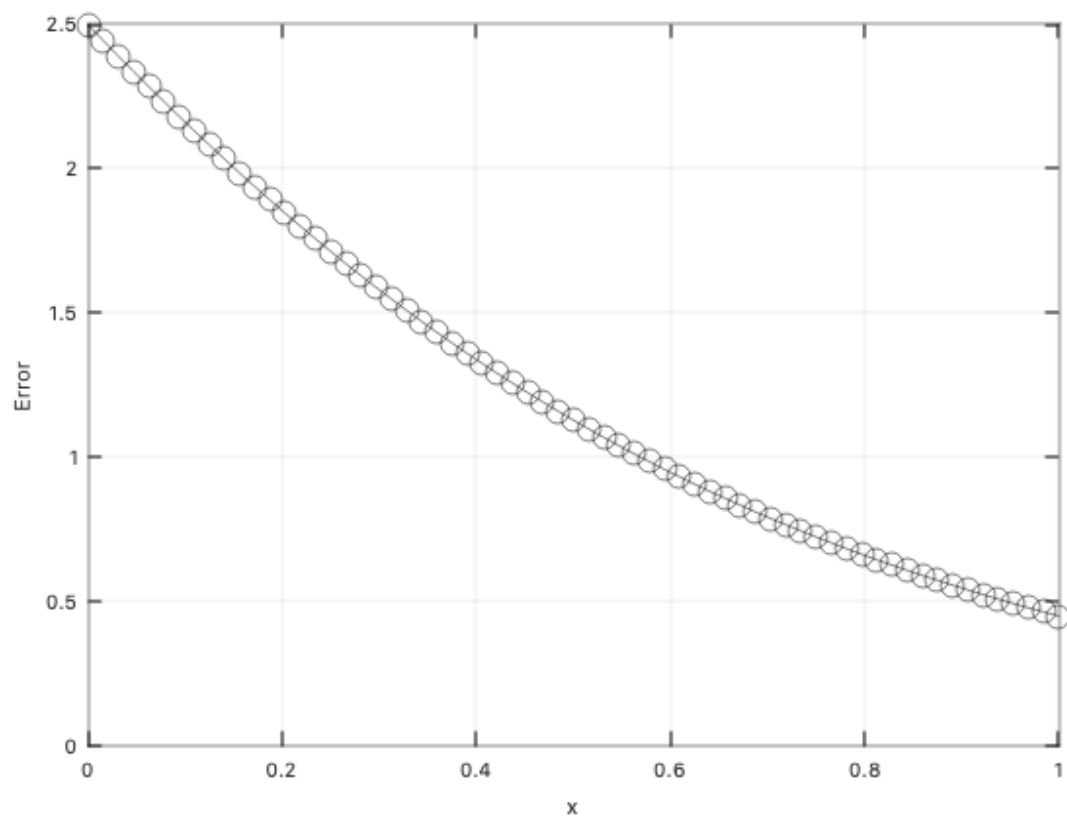


Рис. 6: Распределение оценки погрешности по узлам сетки

Список литературы

- [1] Демидович Б.П., Марон И.А., Шувалова Э.З. Численные методы анализа. — М.: Наука, 1967. — 368 с.
- [2] Амосов А.А., Дубинский Ю.А., Копченова Н.В. Вычислительные методы для инженеров. — М.: Высшая школа, 1994. — 544 с.
- [3] Мудров А.Е. Численные методы для ПЭВМ на языках Бейсик, Фортран и Паскаль. — Томск: МП "РАСКО 1991. — 272 с.
- [4] Самарский А.А., Вабищевич П.Н., Самарская Е.А. Задачи и упражнения по численным методам. — М.: Издательство МГУ, 1989. — 240 с.